

Лабораторная работа №4

“Сравнение методов градиентного бустинга и случайного леса для классификации”

Теоретическая часть

Случайный лес (Random Forest):

Случайный лес — это ансамблевый метод, который использует несколько деревьев решений для улучшения результатов классификации. Каждый элемент случайного леса строится на случайной подвыборке признаков и данных. Это помогает избежать переобучения, так как каждая модель обучается на разных подмножествах данных и признаков.

- **Основные принципы:**
 - Построение множества деревьев решений на случайных подмножествах данных.
 - Голосование или усреднение прогнозов деревьев.
 - Устойчивость к переобучению благодаря случайности.
- **Преимущества:** Хорошо работает с большими объемами данных, не требует нормализации данных.
- **Недостатки:** Может быть менее интерпретируемым, чем одиночное дерево решений.

Градиентный бустинг (Gradient Boosting):

Градиентный бустинг — это метод, в котором множество слабых моделей (обычно деревьев решений) обучаются последовательно. Каждая новая модель исправляет ошибки предыдущей, и этот процесс продолжается до тех пор, пока модель не достигнет желаемой точности.

- **Основные принципы:**
 - Итеративное построение моделей, где каждая новая модель минимизирует ошибку предыдущих моделей.
 - Использует градиентный спуск для оптимизации.
 - Модели обучаются на ошибках предыдущих.
- **Преимущества:** Высокая точность, особенно при большом количестве данных.
- **Недостатки:** Может быть подвержен переобучению, чувствителен к шуму в данных, более долгий процесс обучения.

Основные отличия между случайным лесом и градиентным бустингом:

- **Алгоритм:** Случайный лес строит деревья параллельно, тогда как градиентный бустинг — последовательно.
- **Процесс обучения:** Случайный лес не учитывает ошибки предыдущих деревьев, а градиентный бустинг пытается исправить ошибки на каждом шаге.
- **Скорость обучения:** Градиентный бустинг часто требует больше времени на обучение, чем случайный лес.
- **Подверженность переобучению:** Градиентный бустинг может быть более склонен к переобучению, если не настроены гиперпараметры.

Описание эксперимента

Шаги выполнения:

1. **Загрузка данных:** Используем стандартные наборы данных, например, `Iris` для классификации, `Titanic` или `Wine`.
2. **Предобработка данных:**
 - Очистка данных: удаление или замена пропущенных значений.
 - Кодирование категориальных признаков.
3. **Построение моделей:**
 - Построить модель случайного леса с помощью `RandomForestClassifier` из библиотеки `scikit-learn`.
 - Построить модель градиентного бустинга с помощью `GradientBoostingClassifier`.
4. **Оценка моделей:**
 - Оценить точность моделей с помощью метрик: точность, F1-score, AUC.
 - Применить перекрестную проверку (cross-validation).
5. **Сравнение результатов:**
 - Сравнить метрики моделей и их поведение на тестовой выборке.

Инструменты:

- Python 3.x
- `scikit-learn` — для создания и оценки моделей
- `matplotlib/seaborn` — для визуализации результатов

Пример

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

from sklearn.model_selection import cross_val_score

import matplotlib.pyplot as plt


# Загружаем датасет Iris

data = load_iris()

X = data.data

y = data.target


# Разделяем данные на обучающую и тестовую выборки

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)


# Случайный лес

rf = RandomForestClassifier(random_state=42)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)


# Градиентный бустинг

gb = GradientBoostingClassifier(random_state=42)

gb.fit(X_train, y_train)

gb_pred = gb.predict(X_test)


# Оценка результатов
```

```

print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))

print("Gradient Boosting Accuracy:", accuracy_score(y_test, gb_pred))

print("Random Forest F1-Score:", f1_score(y_test, rf_pred, average='weighted'))

print("Gradient Boosting F1-Score:", f1_score(y_test, gb_pred,
average='weighted'))

# Перекрестная проверка

rf_cv = cross_val_score(rf, X, y, cv=5, scoring='accuracy')

gb_cv = cross_val_score(gb, X, y, cv=5, scoring='accuracy')

print("Random Forest Cross-validation accuracy:", rf_cv.mean())

print("Gradient Boosting Cross-validation accuracy:", gb_cv.mean())

# Визуализация результатов

models = ['Random Forest', 'Gradient Boosting']

accuracy = [accuracy_score(y_test, rf_pred), accuracy_score(y_test, gb_pred)]

plt.bar(models, accuracy)

plt.title("Model Comparison")

plt.ylabel("Accuracy")

plt.show()

```

Этот код:

1. Загружает датасет **Iris**, делит его на обучающую и тестовую выборки.
2. Строит и обучает модели случайного леса и градиентного бустинга.
3. Оценивает точность и F1-score для обеих моделей.
4. Применяет перекрестную проверку и выводит среднюю точность.
5. Визуализирует точность обеих моделей на графике.

Задание

1. Использовать датасет из лаб №2
2. Построить и обучить модель случайного леса для классификации и решить ту же задачу методом градиентного бустинга (с использованием библиотеки `sklearn`)
3. Оценить качество работы моделей с помощью метрик F1, Recall, Precision. Сравнить модели.
4. Загрузить код на гитхаб – (сбросить ссылку преподавателю)